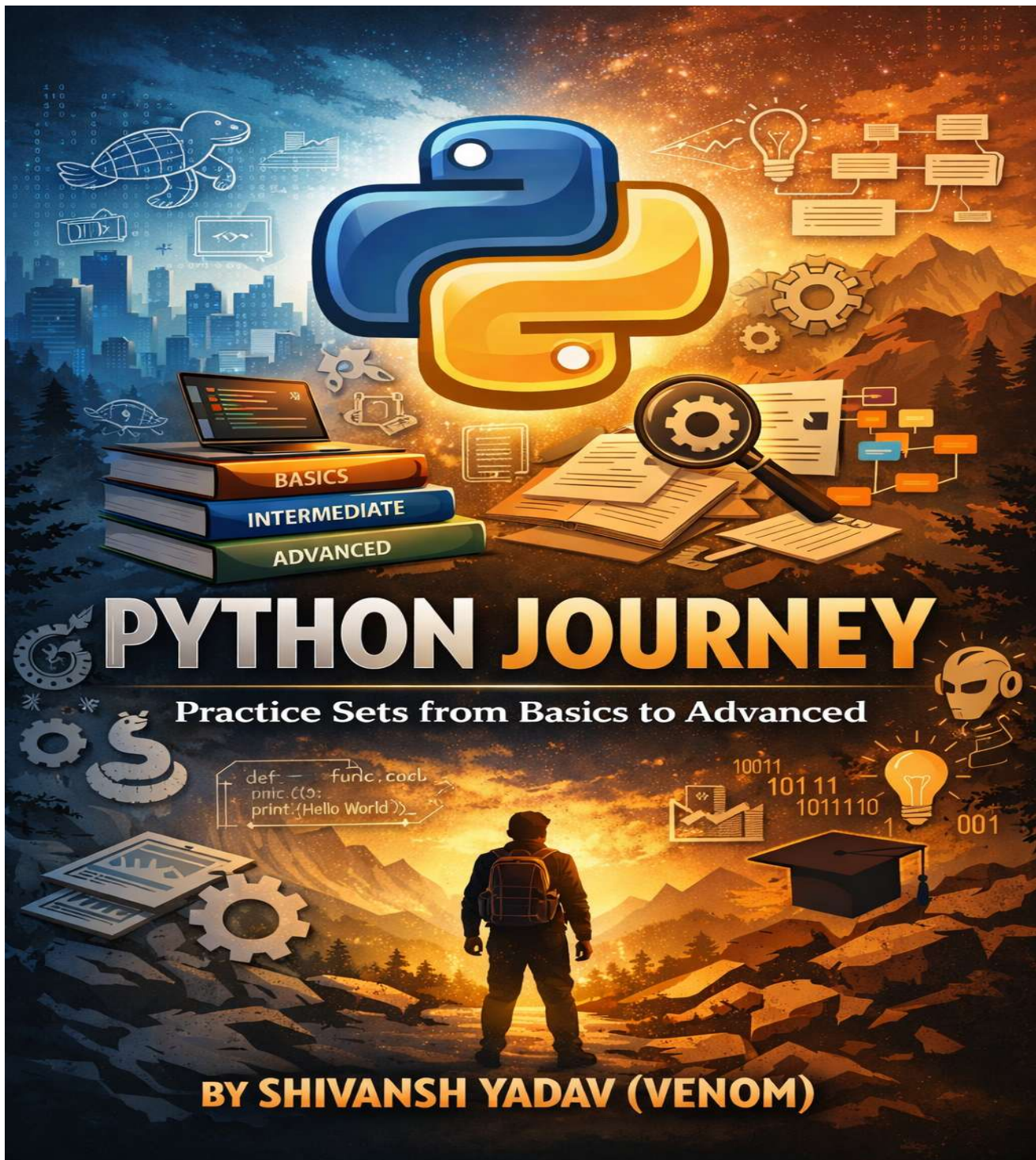


Python Journey – Personal Practice Sets

This document contains my personally curated Python practice sets. The questions are intentionally reworded in a professional manner while preserving the original intent, ensuring conceptual clarity, logical depth, and real-world relevance.



Chapter 1: Modules, Comments & Package Management

Difficulty Level: Easy

Expected Concepts: Learners will gain practical experience in working with multi-line strings, external libraries, filesystem interaction, and code documentation. They will be able to improve program readability, leverage standard modules, and enhance maintainability through proper commenting.

Practice Questions:

1. **Problem 1: Displaying Multi-Line Text Using Triple-Quoted Strings**
Write a Python program that prints a multi-line poem using triple-quoted strings while preserving formatting and line breaks.
2. **Problem 2: Text-to-Speech Using an External Module**
Develop a Python program that uses an external library (such as pyttsx3) to convert text input into audible speech.
3. **Problem 3: Directory Listing Using the os Module**
Create a script that lists all files and folders present in a specified directory using functions from the os module.
4. **Problem 4: Code Documentation Through Comments**
Enhance an existing Python script by adding clear and meaningful comments that explain the purpose of each logical section of the code.

Solution Guidelines:

- Use triple-quoted strings to preserve multi-line formatting.
 - Install and import external modules correctly before usage.
 - Utilize `os` module functions such as directory listing utilities responsibly.
 - Write concise and relevant comments that explain logic rather than restating code.
-

Chapter 2: Variables & Data Types

Difficulty Level: Easy

Expected Concepts: Learners will gain foundational proficiency in handling user input, performing arithmetic operations, and applying comparison logic. They will be able to process numeric data accurately and interpret program output effectively.

Practice Questions:

1. **Problem 1: Sum of Two Numbers**

Write a Python program that accepts two numerical values from the user and calculates their sum, displaying the result clearly.

2. **Problem 2: Remainder Calculation**

Develop a program that computes and displays the remainder obtained when one number is divided by another using the modulo operator.

3. **Problem 3: Input Data Type Identification**

Create a program that accepts user input and determines the data type of the received value using appropriate Python constructs.

4. **Problem 4: Variable Comparison Using Operators**

Implement a program that compares two variables using comparison operators and displays the boolean result of each comparison.

5. **Problem 5: Average of Two Numbers**

Write a program that calculates the average of two user-entered numbers and displays the computed value.

6. **Problem 6: Square of a Number**

Develop a program that computes and displays the square of a number provided by the user.

Solution Guidelines:

- Use appropriate data type conversion for numeric input.
 - Apply arithmetic and comparison operators correctly.
 - Store intermediate results using meaningful variable names.
 - Display outputs in a clear and user-friendly format.
 - Validate inputs where necessary to avoid runtime errors.
-

Chapter 3: Strings

Difficulty Level: Easy–Medium

Expected Concepts: Learners will develop proficiency in Python string manipulation, formatting techniques, and text validation. They will be able to construct dynamic text output, detect formatting issues, and apply escape sequences for structured textual representation.

Practice Questions:

- Problem 1: Greeting Message Using Formatted Strings**
Write a Python program that accepts a user-entered name and displays a greeting message using formatted string techniques.
- Problem 2: Dynamic Letter Template Population**
Develop a program that populates a predefined letter template by inserting a user-provided name and date using string formatting.
- Problem 3: Detection of Consecutive Double Spaces**
Create a program that checks whether a given string contains consecutive double spaces and reports the result accurately.
- Problem 4: Double Space Replacement**
Implement a program that replaces all occurrences of double spaces in a string with single spaces while preserving the remaining content.
- Problem 5: Letter Formatting with Escape Sequences**
Write a program to format a multi-line letter using appropriate escape sequence characters for line breaks and spacing.

Solution Guidelines:

- Use string formatting methods (f-strings, str.format()) consistently.
 - Apply string search and replacement functions for text analysis.
 - Handle whitespace carefully to avoid unintended formatting issues.
 - Use escape sequences (\n, \t) to structure multi-line output.
 - Maintain clarity and readability in formatted text output.
-

Chapter 4: Lists & Tuples

Difficulty Level: Easy–Medium

Expected Concepts: Learners will understand the fundamental behavior of Python's list and tuple data structures, including mutability, ordering, and basic data aggregation. They will be able to collect user input, perform sorting operations, and analyze tuple contents effectively.

Practice Questions:

1. **Problem 1: Storing User-Entered Fruit Names**

Write a Python program that accepts the names of seven fruits from the user and stores them in a list for further processing or display.

2. **Problem 2: Sorting Student Marks**

Develop a program that accepts the marks of six students, stores them in a list, and displays the marks in sorted order.

3. **Problem 3: Demonstrating Tuple Immutability**

Create a program that demonstrates the immutability of tuples by attempting to modify an element and observing the resulting behavior.

4. **Problem 4: Summation of List Elements**

Implement a program to calculate the sum of all numeric elements stored in a list using appropriate iteration or built-in functions.

5. **Problem 5: Counting Zero Values in a Tuple**

Write a program that counts and displays the number of zero values present in a given tuple.

Solution Guidelines:

- Use lists for mutable, ordered data collection.
 - Apply built-in functions such as `sorted()` and `sum()` where appropriate.
 - Demonstrate tuple immutability through controlled code experiments.
 - Use iteration or built-in methods to analyze sequence contents.
 - Maintain clean input handling and readable output formatting.
-

Chapter 5: Dictionaries & Sets

Difficulty Level: Medium

Expected Concepts: Learners will gain a clear understanding of Python's set and dictionary data structures, including uniqueness rules, hashing behavior, and key-value constraints. They will be able to model real-world data using appropriate collection types and predict Python's behavior in edge cases.

Practice Questions:

- Problem 1: Hindi-to-English Dictionary Lookup**
Create a dictionary that maps Hindi words to their English meanings and allows users to retrieve meanings based on input keywords.
- Problem 2: Unique Value Extraction from Inputs**
Develop a program that accepts multiple numeric inputs and displays only the unique values by eliminating duplicates.
- Problem 3: Integer and String Coexistence in a Set**
Write a program to verify whether a Python set can store both an integer and its string representation as distinct elements.
- Problem 4: Set Length with Mixed Data Types**
Determine and display the length of a set containing a combination of numeric and string values, analyzing how Python treats distinct data types.
- Problem 5: Dictionary-Based Preference Storage**
Store friends' favorite programming languages using their names as dictionary keys, ensuring proper key-value mapping.
- Problem 6: Duplicate Keys in a Dictionary**
Analyze and demonstrate the behavior of a dictionary when duplicate keys are defined and explain how Python resolves such conflicts.
- Problem 7: Duplicate Values in a Dictionary**
Examine the behavior of dictionaries when multiple keys are assigned the same value and analyze the resulting data structure.
- Problem 8: Mutability Constraints in Sets**
Evaluate whether mutable objects can be stored inside a set and explain the underlying reasons based on Python's hashing mechanism.

Solution Guidelines:

- Use sets to enforce uniqueness of elements automatically.

- Understand hashing and immutability requirements for set elements and dictionary keys.
 - Demonstrate dictionary behaviour clearly when handling duplicate keys or values.
 - Use meaningful keys and consistent data types for reliable data access.
 - Explain observed behaviour logically rather than relying on assumptions.
-

Chapter 6: Conditional Expressions

Difficulty Level: Medium

Expected Concepts: Learners will develop proficiency in applying conditional statements, comparison logic, and basic string analysis. They will be able to validate inputs, classify data based on rules, and detect patterns within textual content.

Practice Questions:

1. **Problem 1: Largest of Four Numbers**

Write a Python program that accepts four numeric inputs from the user and determines the largest value using conditional logic.

2. **Problem 2: Student Pass/Fail Evaluation**

Develop a program that evaluates whether a student passes based on overall percentage and minimum subject-wise score requirements.

3. **Problem 3: Spam Comment Detection**

Create a program that identifies spam comments by checking for the presence of predefined keywords within a given text.

4. **Problem 4: Username Length Validation**

Implement a program that verifies whether a username contains fewer than ten characters and reports the validation result.

5. **Problem 5: Name Presence Verification**

Write a program to check whether a specific name exists within a given list of names using appropriate search logic.

6. **Problem 6: Grade Assignment System**

Design a program that assigns grades to students based on a defined grading scheme using conditional branching.

7. **Problem 7: Keyword Reference Detection in Text**

Develop a program that analyzes a given post or message to determine whether it references a specified keyword.

Solution Guidelines:

- Use clear and well-structured conditional logic (if, elif, else).
 - Apply comparison and logical operators appropriately.
 - Normalize text (e.g., case handling) before keyword checks.
 - Define grading and validation criteria explicitly to avoid ambiguity.
 - Keep decision logic readable and easy to modify.
-

Chapter 7: Loops

Difficulty Level: Medium

Expected Concepts: Learners will gain strong proficiency in loop constructs, conditional logic, and iterative problem-solving. They will be able to design structured algorithms for numeric computation, pattern generation, and controlled iteration.

Practice Questions:

- Problem 1: Multiplication Table Using a for Loop**
Write a Python program that generates and displays the multiplication table for a given number using a for loop. Ensure the output is structured and easy to read.
- Problem 2: Conditional Greeting Based on Name Prefix**
Develop a program that iterates through a list of names and displays a greeting only for those names that begin with a specified character.
- Problem 3: Multiplication Table Using a while Loop**
Reimplement the multiplication table generation logic using a while loop to demonstrate loop control and iteration management.
- Problem 4: Prime Number Verification**
Create a program that determines whether a given number is prime by checking its divisibility using iterative logic.
- Problem 5: Sum of First n Natural Numbers**
Write a program to compute the sum of the first n natural numbers using loop-based accumulation.
- Problem 6: Factorial Calculation**
Develop a program that calculates the factorial of a given number using iterative logic and proper variable updates.
- Problem 7: Star Pattern Generation Using Loops**
Design a program that generates various star patterns by controlling nested loops and iteration counts.
- Problem 8: Reverse Multiplication Table**
Implement a program that displays the multiplication table of a given number in reverse order, demonstrating loop control and conditional iteration.

Solution Guidelines:

- Choose appropriate loop constructs (for or while) based on problem requirements.
 - Use conditional statements to control iteration flow and filtering logic.
 - Maintain clear loop initialization, update, and termination conditions.
 - Structure output for readability and consistency.
 - Avoid redundant computations by optimizing loop boundaries.
-

Chapter 8: Functions & Recursion

Difficulty Level: Medium

Expected Concepts: Learners will develop proficiency in writing modular and reusable functions, applying recursion effectively, and controlling program output. They will be able to perform data transformation, string manipulation, and pattern generation using well-structured function-based logic.

Practice Questions:

1. **Problem 1: Maximum of Three Numbers**

Design a function that accepts three numeric inputs and returns the maximum value among them. The function should handle comparable numeric data types correctly.

2. **Problem 2: Temperature Conversion (Celsius to Fahrenheit)**

Implement a function that converts a temperature value from Celsius to Fahrenheit using the standard conversion formula and returns the computed result.

3. **Problem 3: Suppressing Newline Behavior in Output**

Demonstrate how to suppress the default newline behavior of Python's `print()` function to control output formatting.

4. **Problem 4: Recursive Sum of Natural Numbers**

Create a recursive function that computes the sum of the first n natural numbers, ensuring a well-defined base case and correct termination.

5. **Problem 5: Decreasing Star Pattern Generation**

Develop a function that displays a decreasing star pattern based on a user-defined number of rows, ensuring clear and readable output formatting.

6. **Problem 6: Measurement Unit Conversion Function**

Write a function that converts values between measurement units (e.g., meters to centimeters or kilograms to grams) using appropriate conversion logic.

7. **Problem 7: Word Removal and Whitespace Cleanup**

Implement a function that removes a specified word from a list of strings and eliminates any trailing or extra whitespace from the resulting elements.

8. **Problem 8: Multiplication Table Generation Using Functions**

Create a function that generates and displays the multiplication table for a given number in a structured and readable format.

Solution Guidelines:

- Define functions with clear input parameters and return values.
 - Use conditional logic and built-in functions where appropriate.
 - Implement recursion with proper base cases to avoid infinite calls.
 - Control output formatting using function arguments and string manipulation.
 - Keep functions focused on a single responsibility for clarity and reuse.
-

Chapter 9: File Handling (Input/Output)

Difficulty Level: Medium–Hard

Expected Concepts: Learners will gain practical expertise in Python file handling, content manipulation, and filesystem operations. They will be able to safely read, modify, compare, and manage files while applying structured logic and error handling.

Practice Questions:

- 1. Problem 1: Keyword Presence Verification in a Text File**
Write a Python program to check whether a specified word exists within a given text file. The program should read file contents safely and report the result accurately.
- 2. Problem 2: High-Score File Management System**
Design a program that maintains a high-score file by reading existing values, comparing them with new scores, and updating the file when a higher score is achieved.
- 3. Problem 3: Multiplication Table File Generation**
Create a Python program that generates multiplication tables for multiple numbers and stores each table in a structured and readable format across one or more files.
- 4. Problem 4: Single-Word File Censorship**
Develop a program that scans a text file and replaces a predefined sensitive word with a placeholder while preserving the remaining content.
- 5. Problem 5: Multi-Word Censorship Implementation**
Extend the censorship logic to handle multiple sensitive words efficiently, ensuring all occurrences are masked consistently.
- 6. Problem 6: Log File Keyword Detection**
Write a program to analyze log files and detect the presence of specific keywords that may indicate warnings, errors, or important events.
- 7. Problem 7: Line Number Identification for Keywords**
Enhance keyword detection by identifying and displaying the exact line number(s) where the specified keyword appears in a file.
- 8. Problem 8: File Duplication Utility**
Implement a Python program that creates an exact duplicate copy of a given file while preserving its content integrity.

9. Problem 9: File Content Comparison

Develop a program to verify whether two files contain identical content by performing a complete content comparison.

10. Problem 10: Programmatic File Content Clearing

Write a Python program to clear the contents of a file without deleting the file itself, ensuring safe and controlled file handling.

11. Problem 11: File Renaming Using Python

Create a program that renames a file using Python's standard library while handling possible errors such as missing files or name conflicts.

Solution Guidelines:

- Use context managers (with statements) for safe file operations.
 - Apply string searching and replacement methods effectively.
 - Handle multiple keywords using iterable data structures.
 - Preserve data integrity during file copying and comparison.
 - Use standard library modules such as `os` and `shutil` for filesystem operations.
 - Implement basic exception handling to manage missing files or access errors.
-
-

Chapter 10: Object-Oriented Programming (OOP)

Difficulty Level: Hard

Expected Concepts: Learners will develop a solid understanding of class design, attribute management, method types, and object interaction. They will be able to distinguish between class-level and instance-level data, apply static methods appropriately, and model real-world systems using object-oriented principles.

Practice Questions:

- 1. Problem 1: Programmer Information Management System**
Design a class to store and manage programmer details within an organization, including attributes such as name, role, and experience. Ensure proper object initialization and controlled data access.
- 2. Problem 2: Calculator Class with Mathematical Operations**
Develop a calculator class that supports mathematical operations including square, cube, and square root. The class should expose methods that perform calculations on user-provided values.
- 3. Problem 3: Class Attributes vs Instance Attributes Demonstration**
Create a program that clearly demonstrates the behavioral and memory differences between class attributes and instance attributes using practical examples.
- 4. Problem 4: Static Method Implementation for User Greeting**
Extend a class by adding a static method that displays a greeting message to users without relying on instance-specific or class-specific data.
- 5. Problem 5: Train Reservation System Modeling**
Model a basic train reservation system using object-oriented principles, including classes for trains, passengers, and ticket booking operations.
- 6. Problem 6: Renaming the self Parameter in Class Methods**
Experiment with renaming the conventional self parameter in class methods and analyze its impact on method behavior and object interaction.

Solution Guidelines:

- Define classes with clear responsibility and meaningful attributes.
- Differentiate class attributes and instance attributes through controlled experiments.

- Implement static methods using the `@staticmethod` decorator where instance data is not required.
 - Apply object-oriented modeling to represent real-world systems logically.
 - Use descriptive method and attribute names to improve code readability.
 - Clearly demonstrate that renaming `self` does not affect functionality, only readability and convention.
-

Chapter 11: Inheritance & Advanced OOP

Difficulty Level: Hard

Expected Concepts: By completing these problems, learners will gain practical mastery of object-oriented programming concepts including inheritance, encapsulation, operator overloading, and special method customization.

Practice Questions:

- 1. Problem 1: Extending a Two-Dimensional Vector to Three Dimensions**
Design a vector class that initially represents two-dimensional vectors and extend it to support three-dimensional vectors using object-oriented principles. Ensure proper initialization and access to all coordinate values.
- 2. Problem 2: Multi-Level Inheritance Using Animals and Pets**
Create a multi-level inheritance hierarchy representing animals and pets, where shared behaviors are defined in base classes and specialized behaviors are implemented in derived classes.
- 3. Problem 3: Employee Salary Management with Property Decorators**
Develop an employee class that manages salary and increment logic using property decorators to control access and validation of salary updates.
- 4. Problem 4: Complex Number Arithmetic via Operator Overloading**
Implement a complex number class that supports arithmetic operations such as addition and subtraction by overloading Python's arithmetic operators.
- 5. Problem 5: Generic N-Dimensional Vector Class**
Create a generic vector class capable of representing vectors in any number of dimensions. The class should support initialization with dynamic dimensions and basic vector operations.
- 6. Problem 6: Custom String Representation for Objects**
Enhance a custom class by overriding built-in string representation methods to provide meaningful and readable object output during printing and debugging.
- 7. Problem 7: Overriding Built-in Length Behavior**
Design a custom class that overrides Python's built-in length behavior to return a meaningful size or count when `len()` is invoked.

Solution Guidelines:

- Apply proper class design principles with clear separation of responsibilities.

- Use inheritance and method overriding only where a true “is-a” relationship exists.
 - Leverage property decorators to enforce validation and encapsulation.
 - Implement operator overloading using appropriate special methods (e.g., `__add__`, `__sub__`).
 - Design classes to support scalability and extensibility (e.g., n-dimensional vectors).
 - Override special methods (`__str__`, `__repr__`, `__len__`) to enhance object usability and readability.
-
-

Chapter 12: Advanced Python Concepts Part - 1

Difficulty Level: Medium–Hard

Expected Concepts: File handling with error recovery, enumerate-based index iteration, list-comprehension-driven data generation, exception-safe arithmetic operations, structured console output, and persistent file-based data storage.

Practice Questions:

- Problem 1: Robust File Reading with Exception Handling**
Write a Python program that attempts to open and read the contents of multiple text files (1.txt, 2.txt, 3.txt). If any file is missing, the program should display a clear warning message for that file without terminating execution, ensuring that remaining files are still processed.
- Problem 2: Index-Based List Processing Using enumerate()**
Write a Python program that iterates over a list using the `enumerate()` function and prints the elements located at the 3rd, 5th, and 7th positions. The solution must rely on index tracking provided by `enumerate()` rather than direct indexing.
- Problem 3: Multiplication Table Generation Using List Comprehension**
Write a Python program that uses a list comprehension to generate the multiplication table (from 1 to 10) for a user-entered integer. Each entry in the list must be a formatted string in the form:
"<number> x <multiplier> = <result>".
- Problem 4: Safe Division with ZeroDivisionError Handling**
Write a Python program that computes the division of two integers (a / b). If the denominator is zero, the program must handle the resulting `ZeroDivisionError` gracefully and display the message **"Infinite"** instead of terminating abruptly.
- Problem 5: Multiplication Table Display and File Persistence**
Write a Python program that generates the multiplication table (1 to 10) for a user-entered integer using a list comprehension. The table must be displayed on the console in a readable format and also stored in a text file named `multiplication_tables.txt`, clearly labeled by the corresponding number.

Solution Guidelines:

- Use structured exception handling with specific exceptions.

- Prefer `enumerate()` for index-value pairing instead of manual counters.
 - Use list comprehensions for concise and readable data generation.
 - Separate data generation, display logic, and file persistence.
 - Use context managers (with statements) for safe file handling.
-

Chapter 13: Advanced Python Concepts Part – 2

Difficulty Level: Medium–Hard

Expected Concepts: Python environment management and reproducibility, input validation with exception handling, functional data processing, list transformation and formatting, dependency snapshot and replication, basic Flask application development and routing

Practice Questions:

1. **Problem 1: Python Virtual Environment Replication**

Create two isolated Python virtual environments. Install a set of required packages in the first environment and demonstrate how to replicate the exact same package configuration in the second environment to ensure consistency across systems or deployments.

2. **Problem 2: Input Validation and Formatted Output**

Write a Python program that accepts a student's name, ranking, percentage score, and phone number as input. Validate that the percentage is between 0 and 100 (inclusive) and the phone number contains exactly 10 digits. If validation fails, display an appropriate error message and terminate execution. On success, display the details using the `str.format()` method in the specified format.

3. **Problem 3: List Transformation and Vertical Formatting**

A list contains the multiplication table of the number 7 as integers. Write a Python program to transform this list into a list of strings, where each element is displayed in vertical format with one value per line.

4. **Problem 4: Filtering Elements Based on Divisibility**

Given a list of integers, write a Python program to filter and extract all elements that are divisible by 5. The program should return a new list containing only those numbers that satisfy the divisibility condition.

5. **Problem 5: Maximum Value Determination Using `reduce()`**

Write a Python program to determine the maximum value in a list of integers using the `reduce()` function. Implement the solution using both a predefined list and a list provided by the user at runtime.

6. **Problem 6: Package Snapshot and Environment Reproducibility**

Using the system Python interpreter, generate a complete list of installed packages and their versions using `pip freeze`. Create a new virtual environment that replicates the same package configuration to ensure reproducibility.

7. Problem 7: Basic Flask Web Application Development

Explore the Flask framework by developing a basic web server application. Initialize a Flask app, define at least one route, and return a response when accessed via a web browser.

Solution Guidelines:

- Use Python virtual environments (venv or virtualenv) to isolate dependencies.
 - Capture and replicate dependencies using pip freeze and requirements files.
 - Apply strict input validation and terminate execution on invalid data.
 - Use exception handling to prevent abrupt program termination.
 - Leverage functional programming tools such as filter() and reduce().
 - Use clear string formatting with str.format() for structured output.
 - Follow Flask best practices for application setup and route definition.
-